人形机器人及其运动控制

王晓芳

清华大学工业工程系

目录

1.人形机器人应用场景
2.人形机器人双足行走的理论基础
2.强化学习原理与PPO算法
3.人形机器人双足行走的强化学习训练
4.人形机器人双足行走 Sim2Real实践

长的像人，就叫人形机器人了吗?

- 从“构成”出发：有类人身体构成的机器人(Human morphologies: 躯干，双手，双腿)
- 从“构成+智能”出发：有类人身体构成和类人智能的机器人



Gu, Zhaoyuan, et al. ***Humanoid Locomotion and Manipulation: Current Progress and Challenges in Control, Planning, and Learning***. arXiv:2501.02116, 3 Jan. 2025.

双足人形—适配人类空间/任务的最佳本体模态

- 人类生存/工作的空间环境，综合考虑人的身体结构、社会习惯和时空需求等因素，设计而成



双足人形，是走向人机共融通用具身智能的最佳本体模态

- 无需对空间环境做重大改动
- 自然地承接了环境/任务，对机器人的复合需求

从任务分类看功能需求

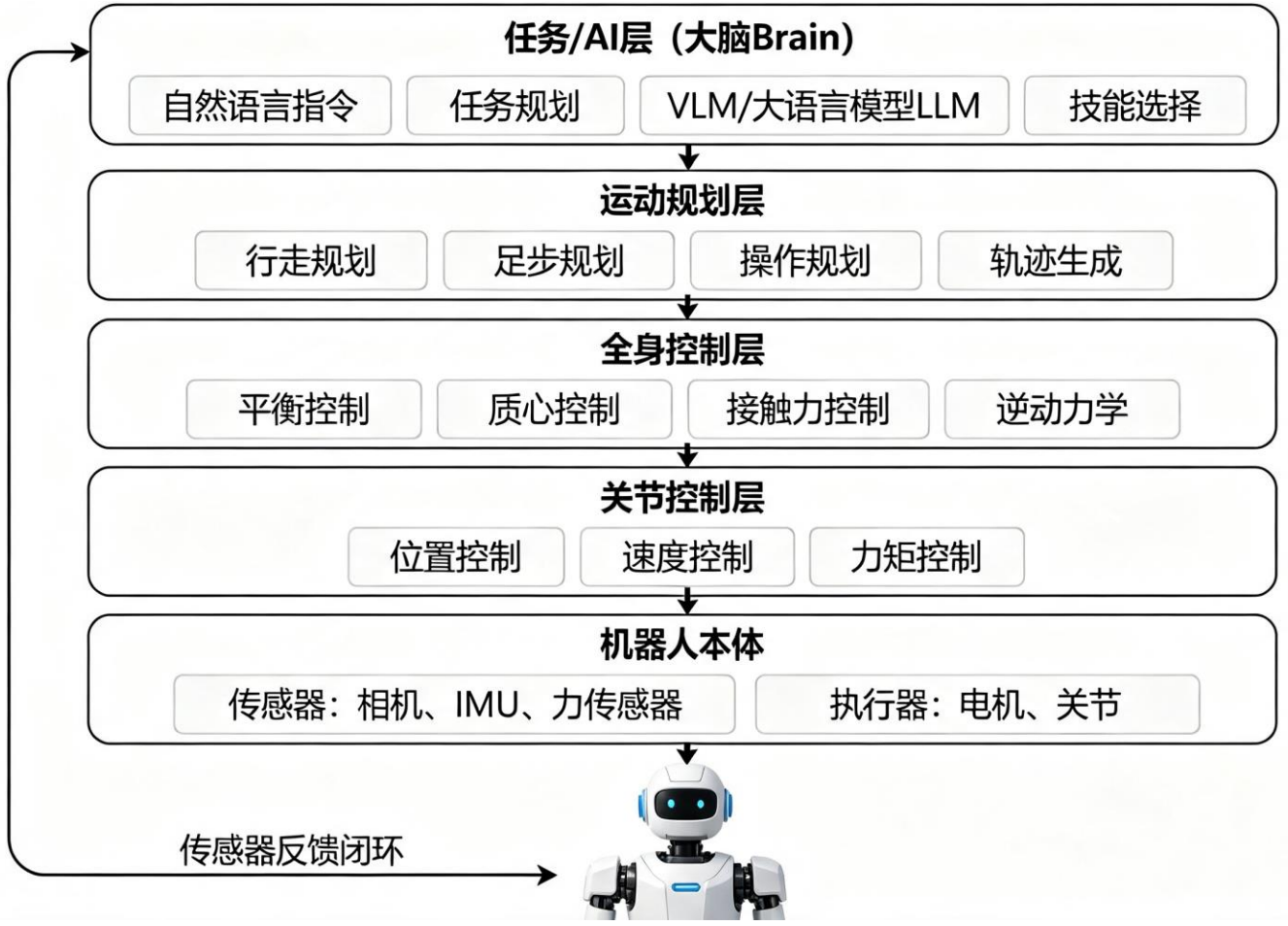
- 任务视角：具体的场景应用，对人形机器人系统提出了哪些共有的功能需求?

应用	移动	感知	接触	操作	交互
表演导览	★★★	★★★	★	★	★★★★★
新零售	★★★	★★★★	★	★★★	★★★★
家庭陪护	★★★★	★★★★	★★★★	★★★	★★★★★
工业巡检	★★★★★	★★★★★	★★	★★★	★★
智能制造	★★★★	★★★★★	★★★★★	★★★★★	★★★
科学发现	★★	★★★★★	★★★★	★★★★★	★★★★

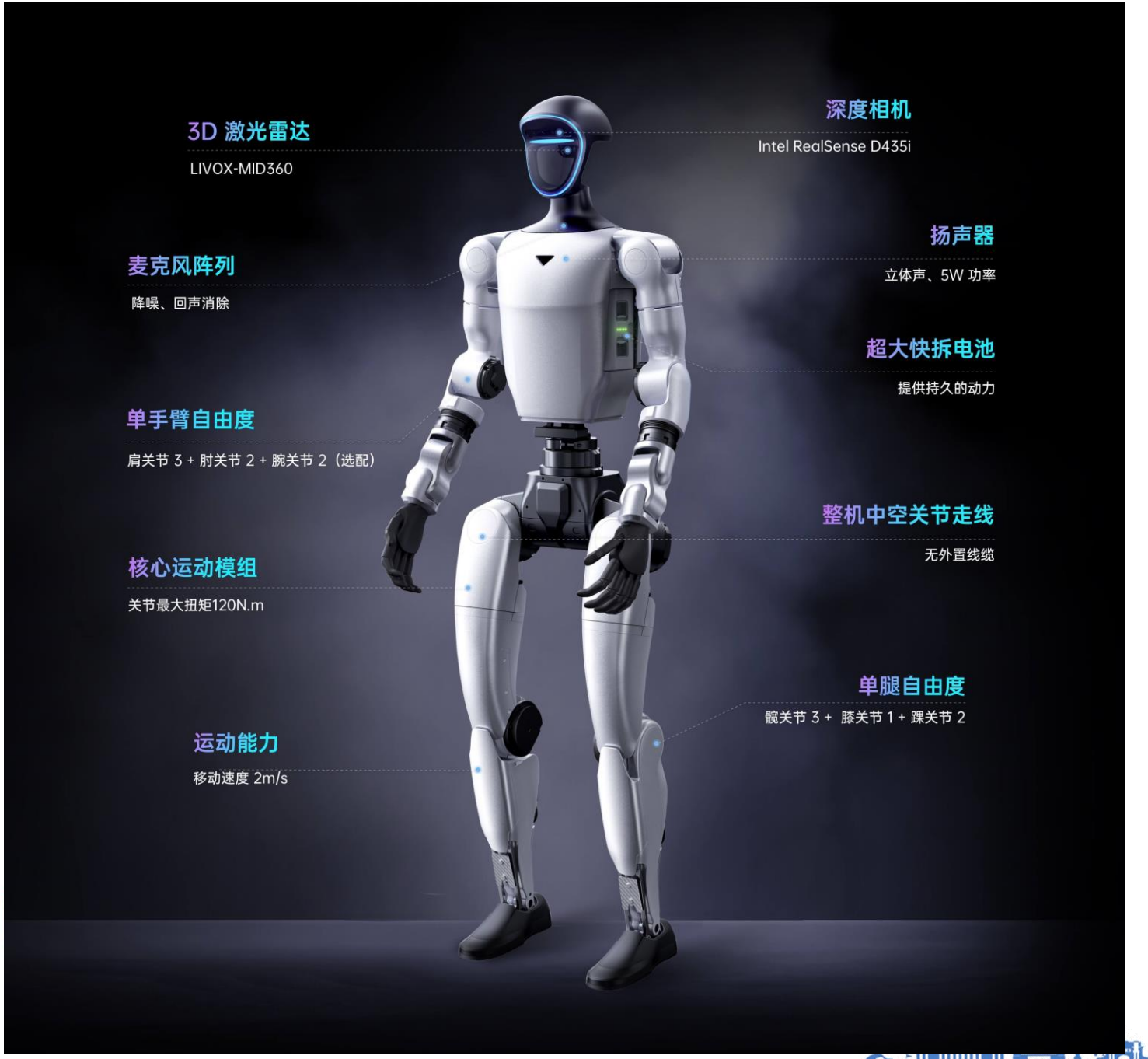
具身智能 Embodied AI

人形机器人智能控制架构

- 任务层→规划层→全身控制→关节控制→电机



宇树G1

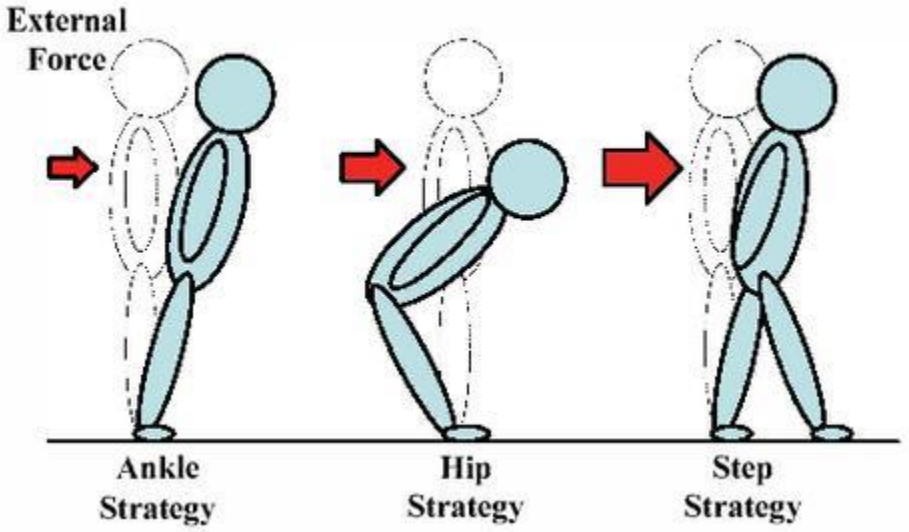

3D 激光雷达
LIVOX-MID360
深度相机
Intel RealSense D435i
麦克风阵列
降噪、回声消除
扬声器
立体声、5W 功率
超大快拆电池
提供持久的动力
单手臂自由度
肩关节 3 + 肘关节 2 + 腕关节 2（选配）
整机中空关节走线
无外置线缆
核心运动模组
关节最大扭矩120N.m
单腿自由度
髋关节 3 + 膝关节 1 + 踝关节 2
运动能力
移动速度 2m/s



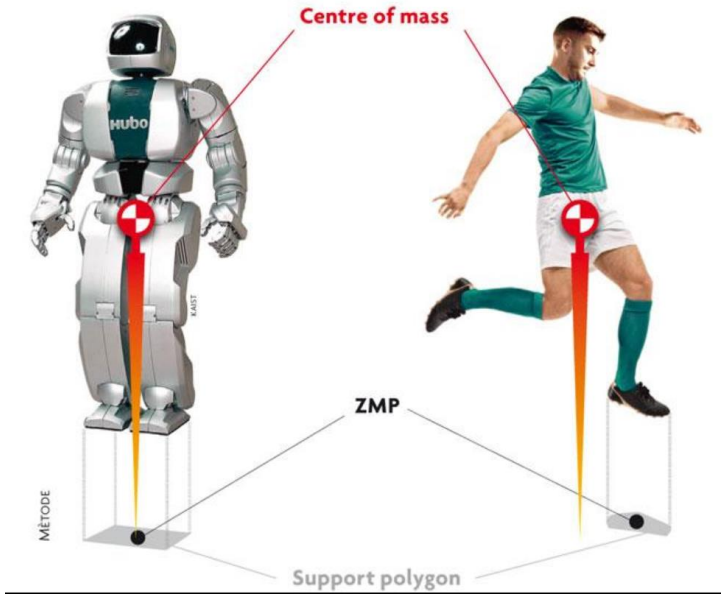
奔跑吧，智能体

为什么双足机器人难控制

- 问题：
  - 平衡
  - 接触
  - 动态运动



	人站立	人形机器人站立
结构	骨骼 + 肌肉 + 肌腱	连杆 + 电机 + 减速器
自由度	约200多个关节自由度（功能运动）	常见20~50 DOF
驱动方式	肌肉主动收缩 + 弹性组织	电机/液压执行器
质量分布	腿部轻、躯干质量集中	电池、电机导致质量集中
柔顺性	天然柔顺	需要控制实现柔顺
能耗	极低（利用肌腱弹性）	较高（持续输出力矩）



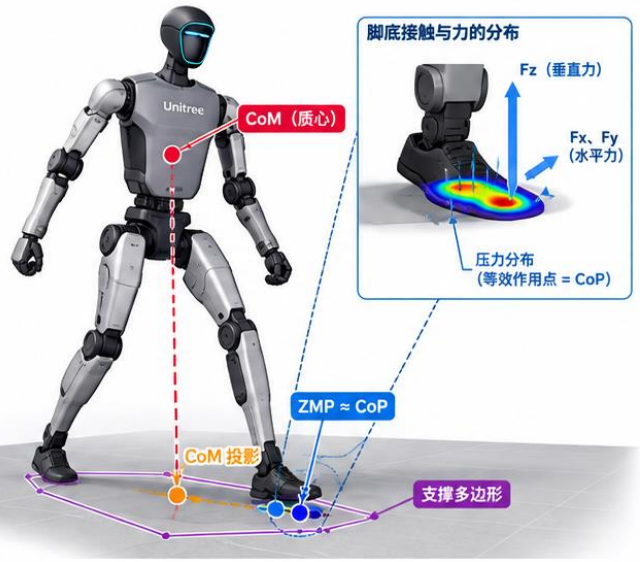
机器人动力学模型

- ZMP（Zero Moment Point，零力矩点）

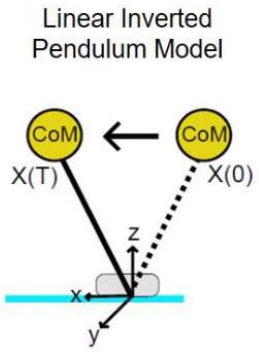
机器人与地面接触时，在地面上存在一个点，使得地面对机器人的水平翻倒力矩为零。

- 对于平地上的双足机器人，可以近似理解成：机器人“踩在地上的受力中心”。

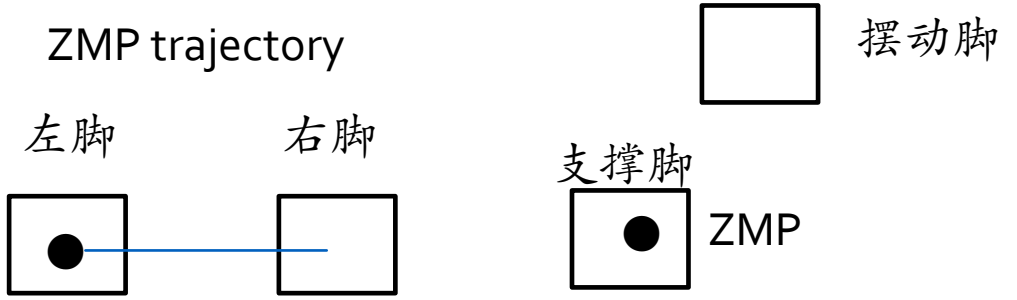
在正常平地接触条件下，ZMP 与 **CoP（Center of Pressure，压力中心）**重合。



线性倒立摆模型

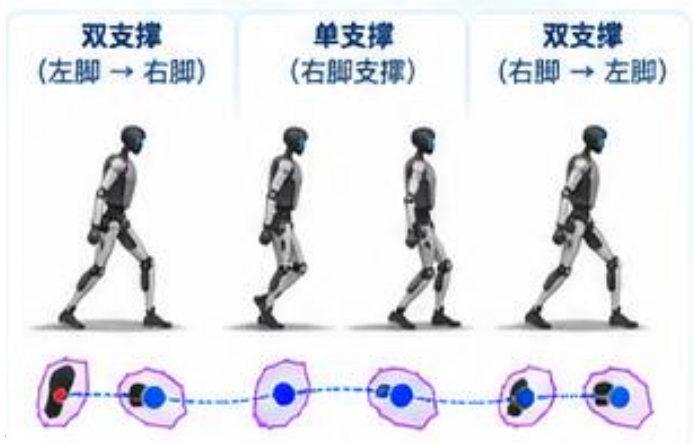


$$ZMP \in SupportPolygon$$



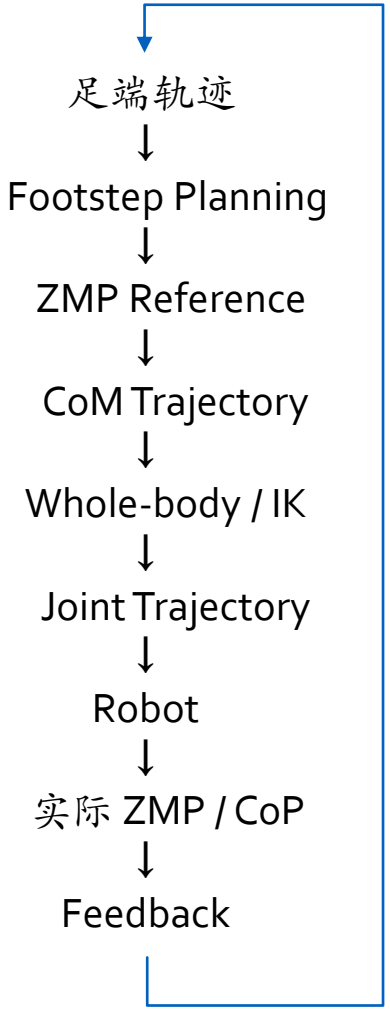
另一只脚落地 → 支撑区域扩大 → ZMP 转移 → 再次单脚支撑。

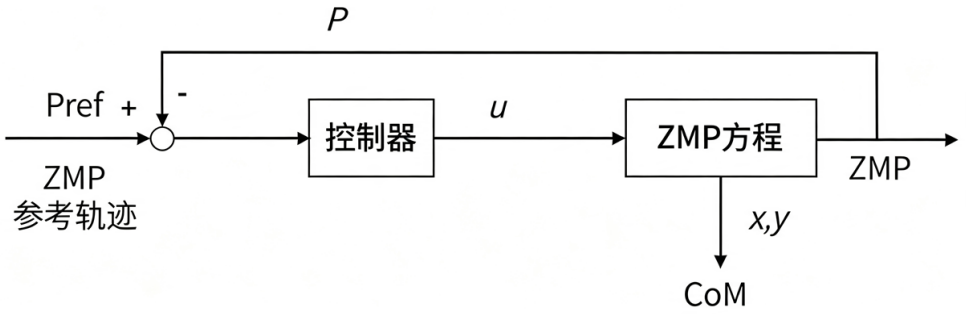




运动控制算法-PID

- PID控制目标：让机器人实际 ZMP 始终跟踪ZMP期望轨迹ZMP_{pref}

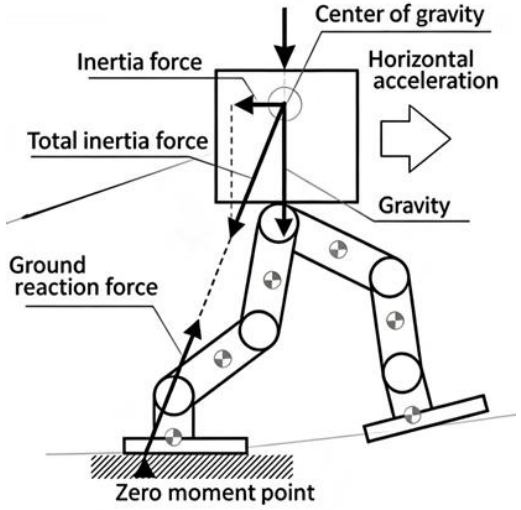




$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \dot{e}(t)$$

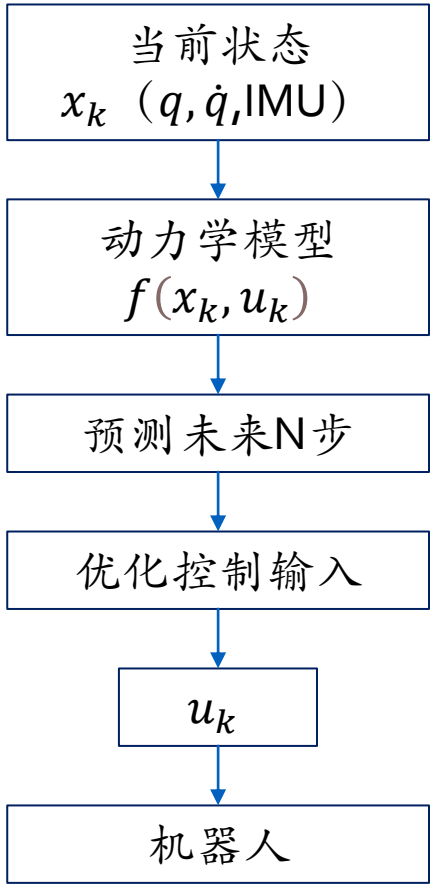
逼近目标　消除长期累积误差　抑制变化趋势，提高稳定性

期望ZMP与实际ZMP差　$ZMP(t) \rightarrow ZMP_{pref}(t)$



运动控制算法-MPC（Model Predictive Control）

- 利用机器人模型预测未来，然后选择最优动作。

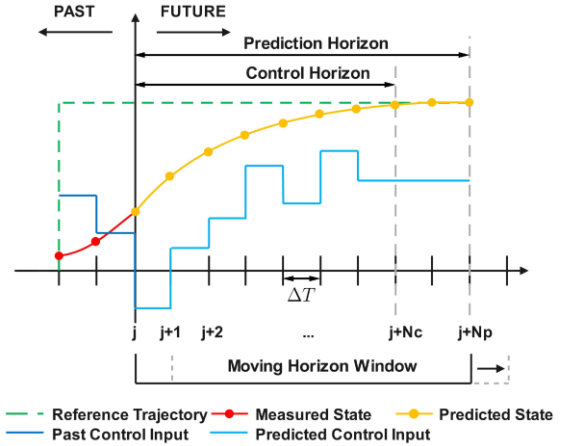


$$x_{k+1} = f(x_k, u_k)$$

$$u = \begin{bmatrix} \tau_1 \\ \tau_2 \\ \vdots \\ \tau_n \end{bmatrix}$$

其中：

- x_k：机器人当前状态
- u_k：控制输入
- $f(\cdot)$：机器人动力学模型



$$J = \sum (x_k - x_{ref})^T Q (x_k - x_{ref}) + u_k^T R u_k$$

s.t.

关节限制：$q_{min} < q(k) < q_{max}$

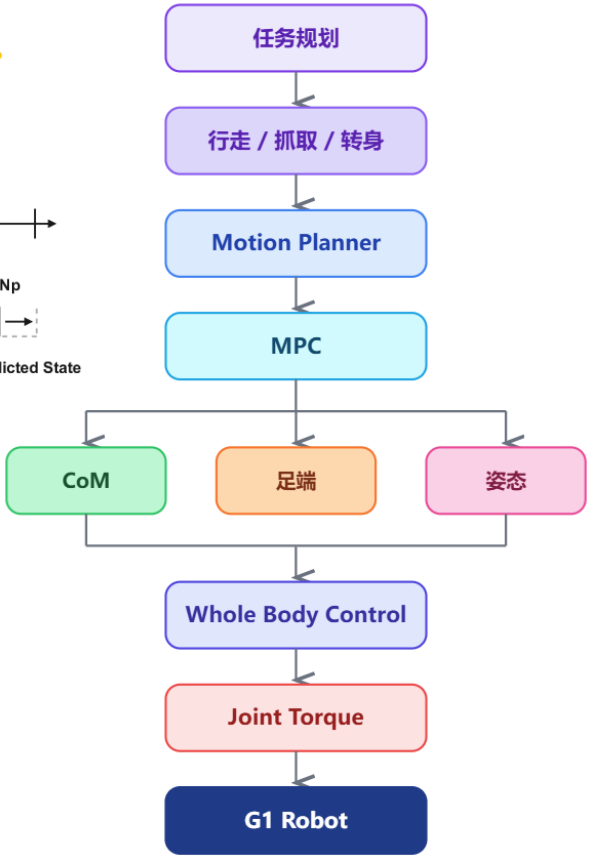
力矩限制：$|\tau(k)| < \tau_{max}$

速度限制：$\dot{q}_{min} < \dot{q}(k) < \dot{q}_{max}$

摩擦力限制：$F_t \leq \mu F_n$

以控制输入u_k（关节力矩）作为优化变量，求解最小化代价；

x_{ref}：系统未来预测状态（质心位置、足部位姿、躯干姿态等）；



运动控制算法-强化学习

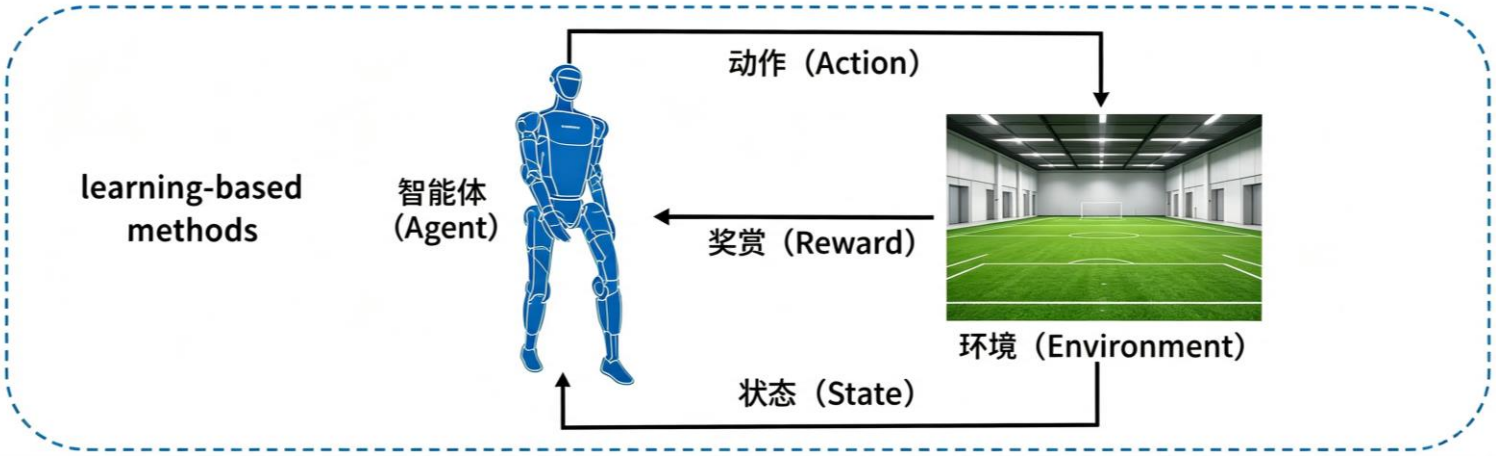
- 传统方法
  - 动力学模型→MPC/WBC→控制量
- 强化学习
  - 状态→策略网络→控制量
  - $s_t \rightarrow \pi_\theta(a_t|s_t) \rightarrow a_t$

“现在算一下应该怎么做”

↓

以前已经学会了遇到这种情况应该怎么

让机器人通过大量“试错”，学习一个从状态到控制动作的映射。



Observation
- Joint position
- Joint velocity
- IMU
- CoM
- Foot contact velocity

↓

RL

↓

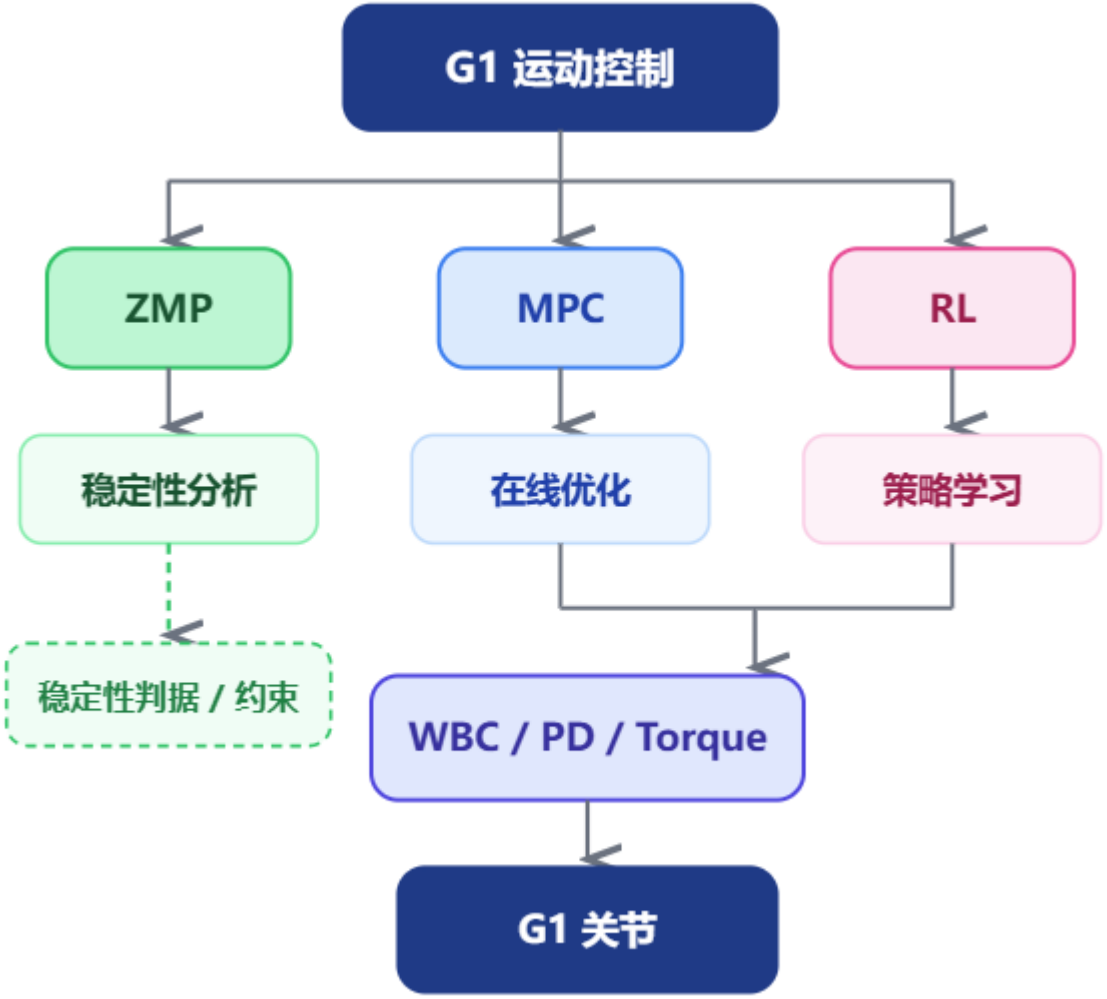
Joint / Torque

↓

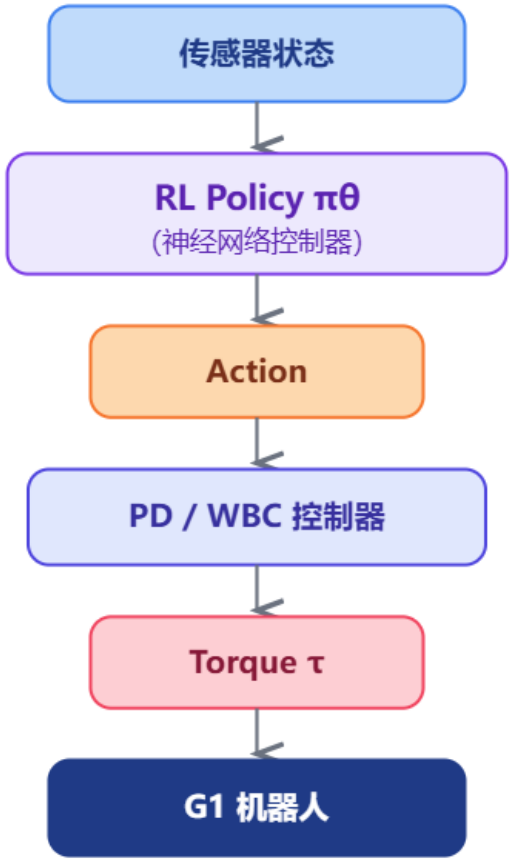
G1 Robot

运动控制算法比较

- ZMP：是否稳定?
- MPC：未来几步怎么控制?
- RL：面对这种状态，我学会了怎么动作?
- WBC：如何把任务转化成全身协调的关节控制?



G1强化学习控制的核心框架



强化学习输出的： a

通常不是直接电机力矩：$a = \Delta q$

即：关节目标增量：$q_d = q_{default} + a$

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q})$$

最终得到：τ

强化学习如何描述机器人?

强化学习有三个核心：(S,A,R)

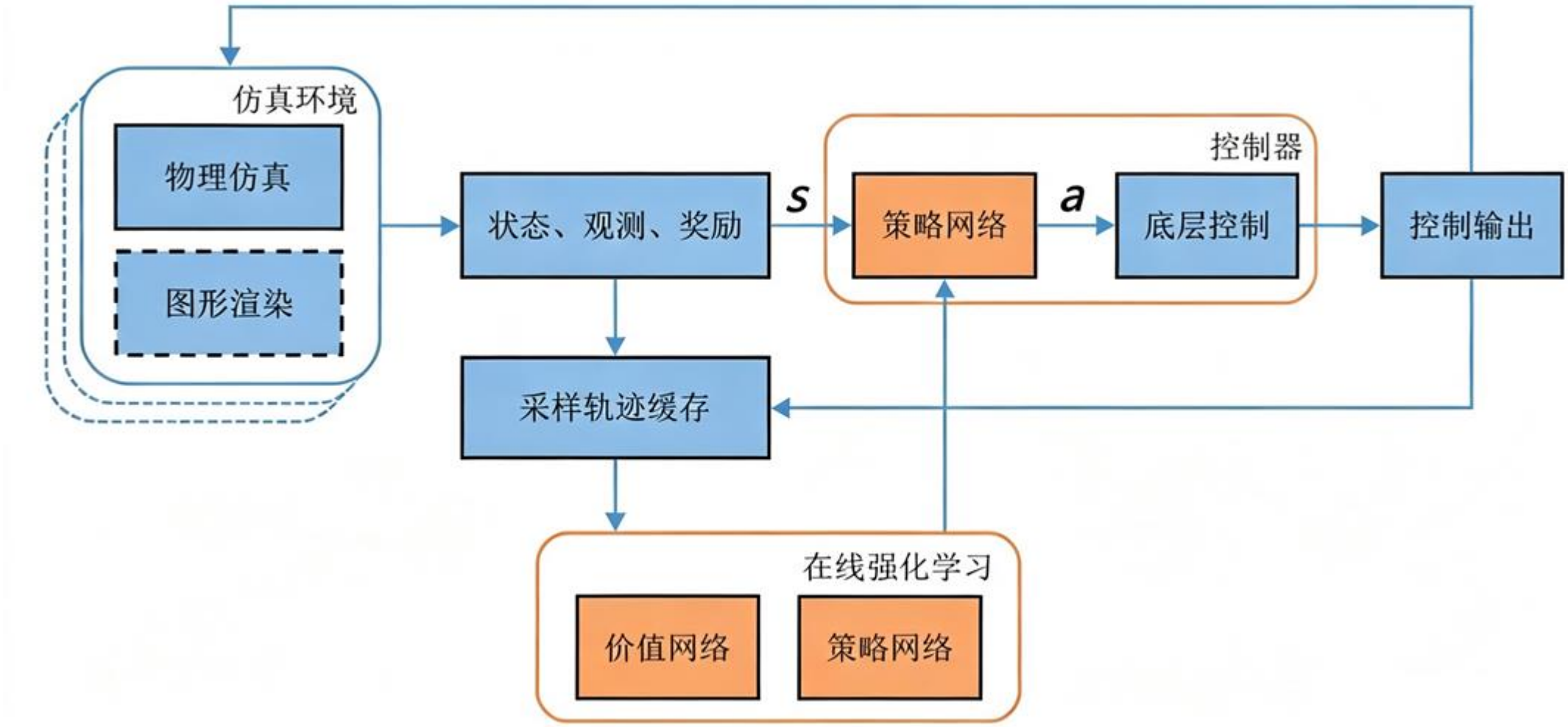
- State 状态 s
  - 机器人本体 $s = [q, \dot{q}, v, \omega, g, v_{cmd}]$
    - 关节角q：$q_1, q_2, \ldots, q_{29}$
    - 关节速度:$\dot{q}$
  - 基座状态ω: 身体高度 ,姿态 ,角速度 ,线速度

- Action 动作 $\boldsymbol{a}$
  - 29自由度 G1：$a \in R^{29}$
  - 每个关节应该如何调整
  - 例如：$a = [0.02, -0.01, \ldots]$

- Reward 奖励
  - 速度奖励
    - 希望：$\boldsymbol{v_x \rightarrow v_{cmd}}$
    - 奖励：$\boldsymbol{r_v = -|v_x - v_{cmd}|}$
  - 姿态稳定奖励
    - 身体不要倾斜：$\boldsymbol{r_{stable} = -\theta^2}$
  - 能耗惩罚
    - 避免大力矩：$\boldsymbol{r_\tau = -\tau^2}$

- $R = w_1 \boldsymbol{r_v} + w_2 \boldsymbol{r_{stable}} + w_3 \boldsymbol{r_\tau}$

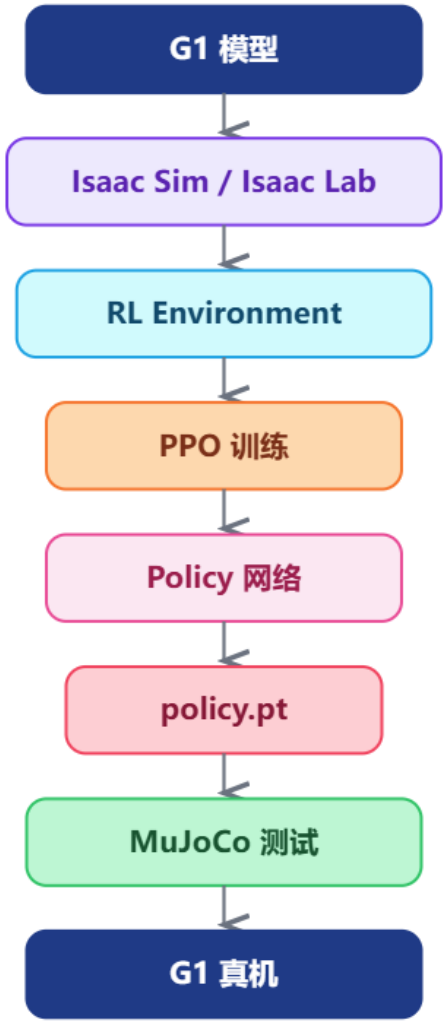
强化学习模型训练与验证框架



Unitree RL Lab中的训练流程

- Unitree RL Lab 基于 Isaac Lab，训练入口通常是：
  - scripts/rsl_rl/train.py
- 推理：
  - scripts/rsl_rl/play.py
- 训练和部署流程包括：
  - Isaac Lab训练
  - 策略验证
  - MuJoCo Sim2Sim
  - Sim2Real真机部署
- 为什么需要 Isaac Lab?
  - 强化学习最大的困难：需要大量数据。
  - 真实机器人：一次摔倒：时间长 ,有风险

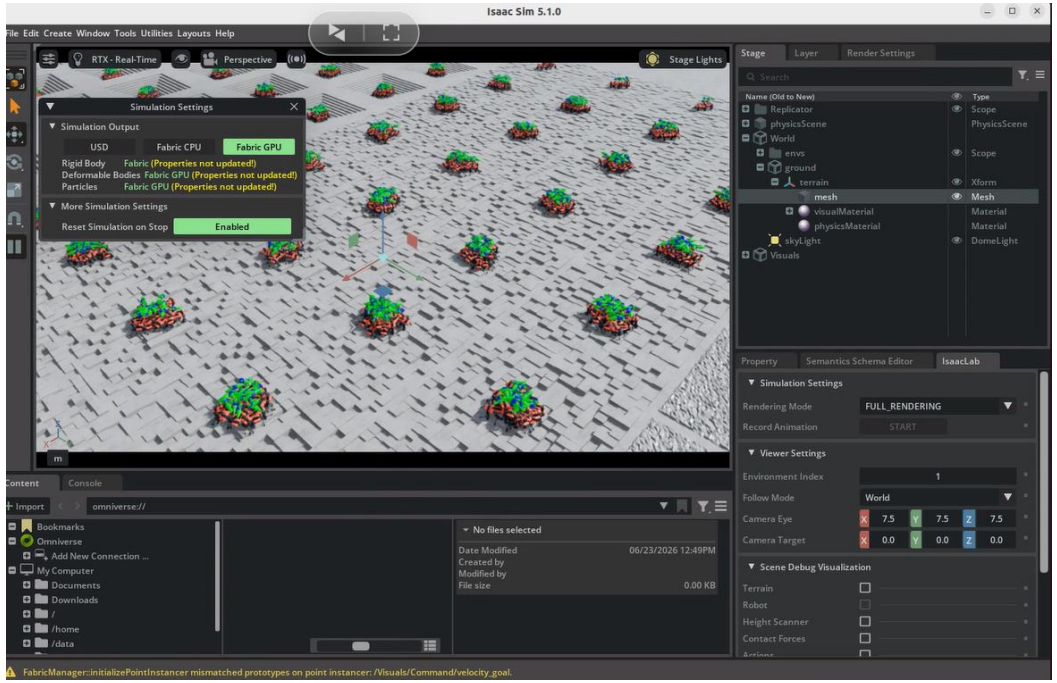
```
unitree_rl_lab/
├── source/unitree_rl_lab/unitree_rl_lab/
│   ├── assets/robots/          # 机器人模型配置
│   │   ├── unitree.py          # ArticulationCfg 定义
│   │   └── unitree_actuators.py
│   ├── tasks/                  # 任务环境
│   │   ├── velocity/           # 速度跟踪
│   │   └── mimic/              # 动作模仿
│   └── utils/mdp/              # MDP 组件
│       ├── observations.py
│       ├── actions.py
│       └── rewards.py
├── scripts/rsl_rl/
│   ├── train.py                # 训练入口
│   └── play.py                 # 推理入口
├── deploy/
│   ├── common/                 # 共享运行时
│   │   ├── isaaclab/           # ManagerBasedRLEnv
│   │   └── FSM/                # 状态机框架
│   └── robots/
│       ├── g1_29dof/           # G1 控制器
│       ├── go2/                # Go2 控制器
│       └── h1/                 # H1 控制器
└── unitree_rl_lab.sh           # 安装/运行 CLI 包装
```

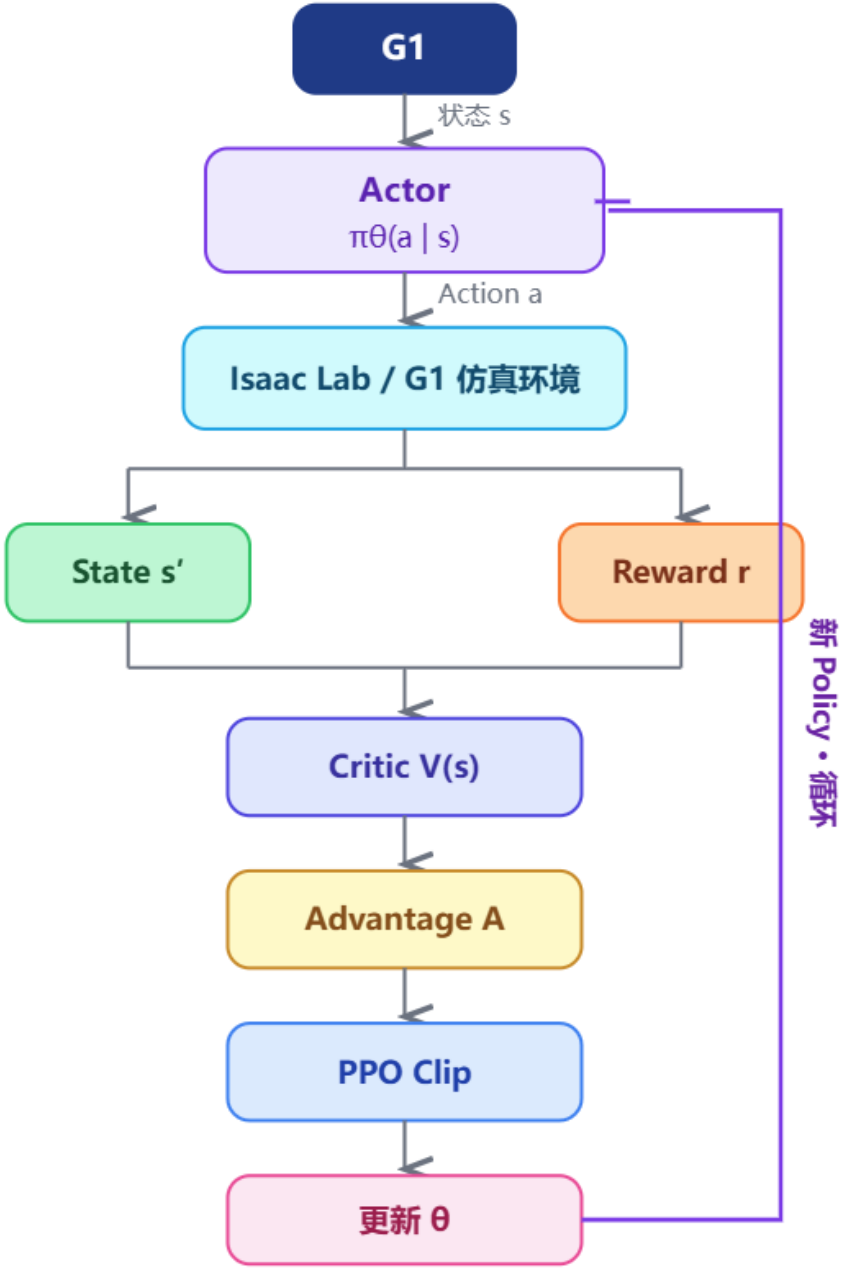


为什么需要 Isaac Lab?

- 强化学习最大的困难：需要大量数据。
- 真实机器人：一次摔倒：时间长 ,有风险 ,成本高

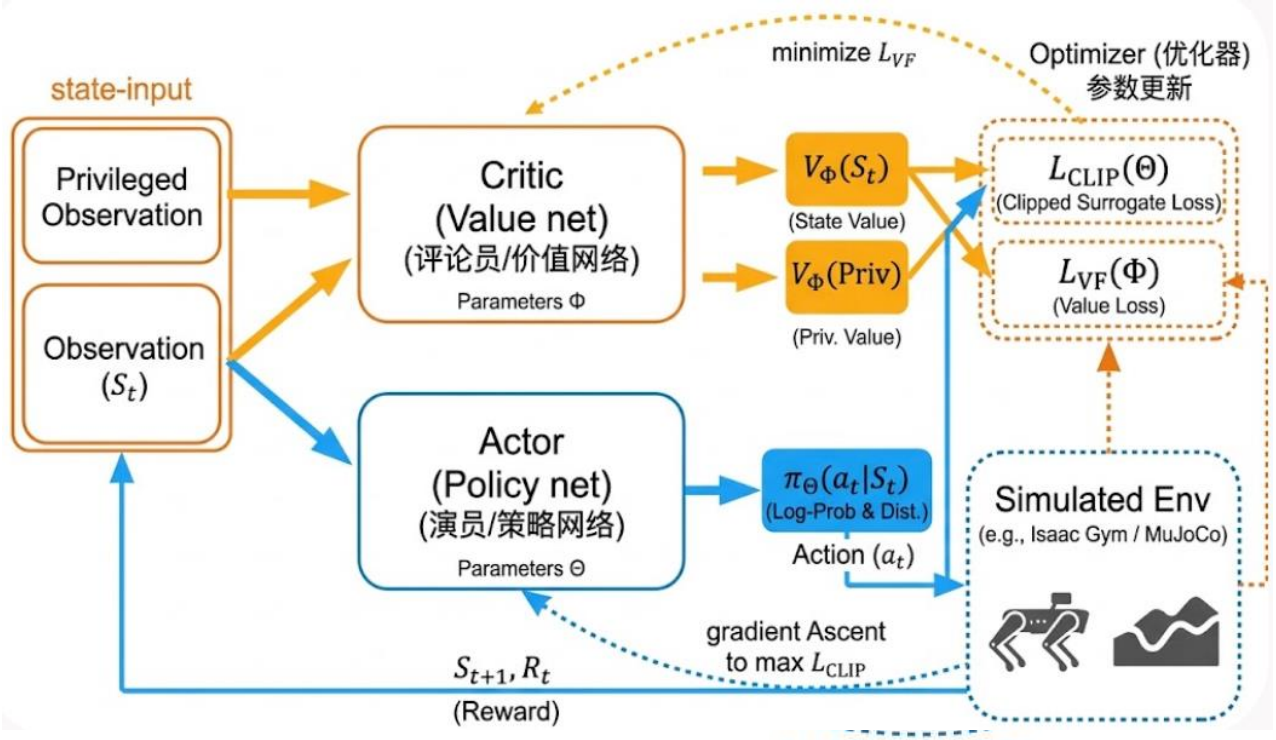
- Isaac Lab同时训练：4096个机器人环境。
- 每秒产生大量数据：(s,a,r,s') 更新策略。





PPO在这里干什么?

- Unitree RL Lab 常用 PPO（Proximal Policy Optimization）进行策略训练。
  - Actor: 仅接收传感器可实测数据（如 IMU 姿态、角速度、关节角度/速度、历史动作命令）。
  - Critic: 额外接收特权物理参数（地面摩擦、刚体质量扰动、接触状态、阻尼），大幅提升价值估计准确度



$$A(s_t, a_t) = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$L_{VF} = E\left[\left(V_\phi(s) - \left(r + \gamma V_\phi(s')\right)\right)^2\right]$$

$$L_{CLIP}(\theta) = -E[\min(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$$

$$\boxed{L = L_{policy} + c_v L_{value} - c_e H}$$

Actor:根据当前state，生成下一个action
Critic:根据当前state，预测执行一个action之后的reward值

为什么要 Clip?

- **限制策略更新幅度**，避免单次更新中策略变化过大

$$J_{PPO}(\theta) = E[min(r_t(\theta)A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)]$$

最小值，确保始终选择更保守的估计，防止过度优化

裁剪操作，防止策略变化过大

$r_t(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}$ 重要性采样权重，表示同一个动作在新旧策略下的概论比例

$\pi_\theta(a|s)$：新策略（待优化网络），在状态s选择动作a的概率

$\pi_{old}(a|s)$：旧策略（采集轨迹数据时冻结的策略网络）

$A_t = A^{\pi_{\theta}old}(s,a)$旧策略下的优势函数估计

让策略变好，但不要一次改变太多。

$$A_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

深度强化学习算法--PPO算法

Algorithm 1 PPO-Clip

Input: initial policy parameters θ_0, initial value function parameters ϕ_0

for $k = 0, 1, 2, \ldots$ **do**

Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.

Compute rewards-to-go $\hat{R}_t$.

Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.

Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.

Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

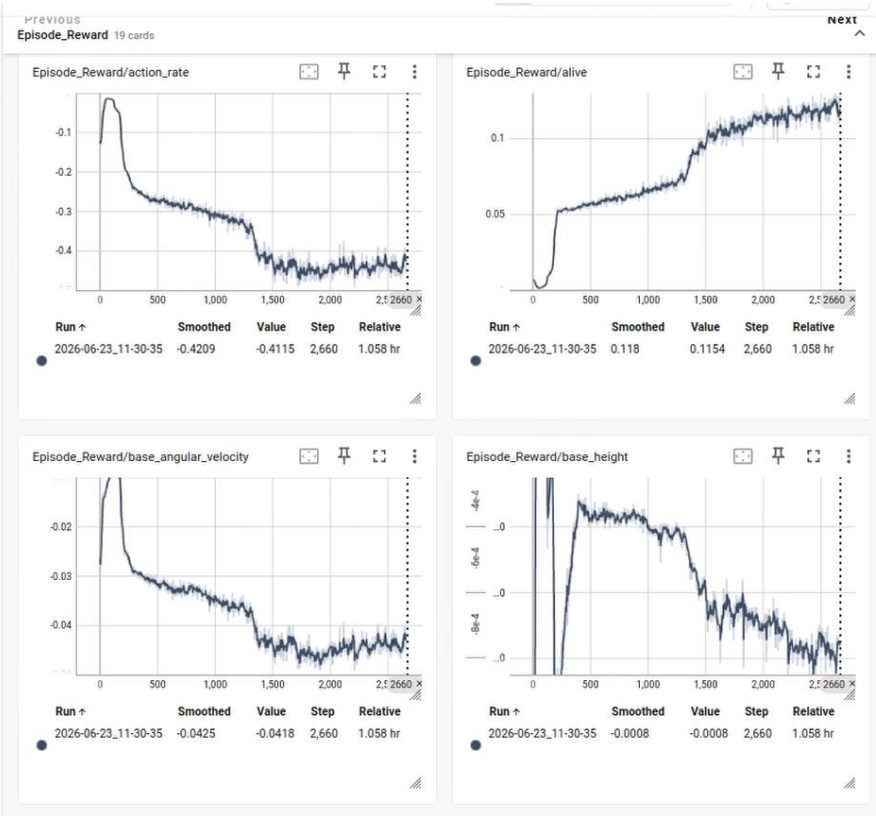
typically via some gradient descent algorithm.

end for

资料来源:https://spinningup.openai.com/en/latest/algorithms/ppo.html

训练过程 Reward 曲线





训练效果演示验证

- 如果想要在 Gym 中查看训练效果，可以运行以下命令

python legged_gym/scripts/play.py--task=xxx

- 说明:
  - Play 启动参数与 Train 相同。
  - 默认加载实验文件夹上次运行的最后一个模型。
  - 可通过 load run 和 checkpoint 指定其他模型。
- 导出网络:
- Play 会导出Actor 网络，保存于 logs/{experiment_name}/exported/policies 中:
  - 普通网络(MLP)导出为 policy_1.pt
  - RNN 网络，导出为 policy_lstm_1.pt
- 导HONNX

注:ONNX(Open Neural Network Exchange)是一种针对机器学习所设计的开放式的文件格式，用于存储训练好的模型。

Sim-to-Real 迁移问题

- 强调仿真成功不代表真实成功

偏差类型	典型来源	对运控的影响
系统参数偏差	质量/惯量/摩擦/阻 尼	步态相位错、恢复能力下降
接触模型偏差	足底接触、碰撞恢复、 地面顺应性	脚滑、绊倒、落脚失真
观测链路偏差	噪声、延迟、漂移、**丢帧**	状态估计变差、闭环控制效果变差
执行器偏差	死区、饱和、温升、 频率限制	输出跟不上、动作抖动
任务分布偏差	地形、载荷、环境结 构不一致	仿真高分，真机失效

实验：行走参数探索

- 实验1：改变 Command
- 实验2：改变 Reward



清華大學
Tsinghua University

谢谢！